SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA1220	**Course Title:**	Computer Architecture
CO Assessed:	CO2 – Precision (BL3)	**Assessment:**	Debugging and Optimisation task
Weightage:	45%	**Total Marks:**	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems.		
Student Name:	**LOKESH S**	**Reg. No.:**	**192572151**
Section / Batch:	**CSE(AI) / 2ND YEAR**	**Date:**	**05/08/2026**

Instructions to Students

- Identify arithmetic errors and performance bottlenecks in the given problem.
- Analyse the execution steps to locate the source of errors.
- Debug the algorithm by correcting logical and computational faults.
- Optimize the solution using appropriate arithmetic techniques or algorithms.
- Evaluate the optimized solution for correctness, accuracy, and efficiency.

QUESTIONS

1. A computer system performing signed binary subtraction using 2's complement produces incorrect results for certain operand combinations. Debug the subtraction process by examining binary conversion, complement generation, and overflow detection. Suggest appropriate corrections to ensure accurate signed arithmetic.
2. A digital processor implementing a carry look-ahead adder fails to produce the correct sum for some input combinations. Debug the generate and propagate logic to identify the source of the error. Recommend modifications that restore correct operation while maintaining high-speed performance.
3. A hardware multiplier using Booth's algorithm consumes more clock cycles than expected during signed multiplication. Debug the execution sequence by analyzing bit-pair decisions, arithmetic shifts, and register updates. Propose optimization techniques to improve execution efficiency without affecting accuracy.
4. A processor implementing the non-restoring division algorithm produces incorrect quotient values for specific dividend and divisor combinations. Debug the step-by-step division process by examining partial remainders, shifting operations, and quotient-bit generation. Suggest corrections to obtain accurate division results.
5. A graphics processing application experiences performance degradation due to frequent floating point computations using IEEE 754 double-precision representation. Debug the arithmetic operations to identify unnecessary precision or computational overhead. Recommend suitable optimization techniques to improve execution speed while maintaining the required numerical accuracy.

Code of Conduct

*I **LOKESH S (192572151)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic*

integrity rules will result in disciplinary action.

Signature of the Student: LOKESH S

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Identification of Errors / Bugs	20	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

Marks Evaluation:

Q. No	Identification of Errors / Bugs (20)	Debugging Approach & Analysis (20)	Correctness of Debugged Solution (25)	Optimization Techniques Applied (20)	Testing and Documentation (15)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Course Coordinator Faculty Incharge

1. SIGNED BINARY SUBTRACTION USING 2'S COMPLEMENT

Problem

A computer system performing signed binary subtraction using 2's complement produces incorrect results for certain operand combinations

due to errors in binary conversion, complement generation, or overflow detection.

Debugging Steps

1. Verify that both operands are converted correctly into binary.
2. Ensure both numbers use the same number of bits (sign extension if necessary).
3. Generate the 2's complement of the subtrahend correctly (invert bits and add 1).
4. Perform binary addition instead of subtraction.
5. Ignore the final carry-out.
6. Check the sign of the result.
7. Detect overflow.

Solving Steps
Example: 7 − 3
7 = 0111
3 = 0011

2's complement of 3

0011
1100
+0001

1101

Addition

 0111
+ 1101

---
1 0100

Result = 0100 = 4

Detect Overflow
Overflow occurs when:

- Two positive numbers produce a negative result.
- Two negative numbers produce a positive result.

Overflow Condition:

Carry into MSB ≠ Carry out of MSB
Example:
0111
+0011

1010
Overflow = Yes

Result
Correct signed subtraction is achieved by proper binary conversion, correct 2's complement generation, and overflow detection.

2. CARRY LOOK-AHEAD ADDER (CLA)

Problem
A Carry Look-Ahead Adder fails to produce the correct sum because of errors

in Generate (G) and Propagate (P) logic.

Debugging Steps

1. Verify Generate logic.
2. Verify Propagate logic.
3. Check carry equations.
4. Test XOR outputs.
5. Verify wiring connections.
6. Test all input combinations.

Solving Steps

Given:

A = 1010

B = 0101

Cin = 0

Generate:

$G_i = A_i \times B_i$

Propagate:

$P_i = A_i \oplus B_i$

Carry:

$C_1 = G_0 + P_0 C_{in}$

$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_{in}$

$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_{in}$

Sum:

$S_i = P_i \oplus C_i$

Detect Overflow

Overflow occurs if:

Carry into MSB ≠ Carry out of MSB

- Positive + Positive = Negative
- Negative + Negative = Positive

Result

Correct Generate and Propagate logic restores accurate and high-speed addition.

3. BOOTH'S MULTIPLICATION ALGORITHM

Problem

A Booth multiplier consumes more clock cycles than expected during signed

multiplication.

Debugging Steps

1. Verify Booth bit pair (Q ,Q).
2. Check Add/Subtract operation.
3. Verify arithmetic right shift.
4. Verify accumulator update.
5. Verify multiplier register.
6. Verify counter decrement.
7. Repeat until counter becomes zero.

Solving Steps
Example:
Multiplicand = 5 = 0101
Multiplier = 3 = 0011
Booth Rules

Q Q	Operation
00	No Operation
01	Add M
10	Subtract M
11	No Operation

Each cycle:

- Check bit pair.
- Perform operation.
- Arithmetic right shift.
- Update registers.
- Decrement counter.

Final Product:
00001111 = 15

Detect Overflow
Overflow occurs when:

- Product requires more bits than available.
- Sign extension becomes incorrect.

  Check whether the product exceeds the register size.

  Result

  Correct Booth decisions, arithmetic shifts, and register updates reduce execution cycles while maintaining accurate signed multiplication.

4. NON-RESTORING DIVISION ALGORITHM

Problem

A processor implementing the non-restoring division algorithm produces

incorrect quotient values.

Debugging Steps

1. Verify dividend and divisor loading.
2. Check left shift operation.
3. Verify subtraction/addition.
4. Check partial remainder.
5. Generate quotient bit correctly.
6. Restore remainder if necessary.
7. Verify final quotient and remainder.

Solving Steps
Example:
Dividend = 13 (1101)

Divisor = 3 (0011)
Steps:

1. Shift left.
2. Subtract divisor.
3. If remainder ≥ 0
   Quotient bit = 1
   Else
   Add divisor

   Quotient bit = 0
   Repeat until all bits are processed.
   Final Answer
   Quotient = 0100 = 4

   Remainder = 0001 = 1

Detect Overflow
Overflow occurs when:

- Divisor = 0 (Division by zero).

- Dividend exceeds register capacity.
- Partial remainder exceeds available bits.

Result

Correct shifting, remainder update, and quotient generation ensure accurate division results.

5. IEEE 754 DOUBLE-PRECISION FLOATING-POINT ARITHMETIC

Problem

A graphics processing application experiences performance degradation due to

frequent IEEE 754 double-precision floating-point computations.

Debugging Steps

1. Check whether double precision is really required.
2. Identify repeated floating-point operations.
3. Check unnecessary type conversions.
4. Analyze normalization and rounding operations.
5. Measure floating-point execution time.
6. Check compiler optimization settings.

Solving Steps

Optimization:

- Replace double (64-bit) with float (32-bit) where sufficient.
- Use Fused Multiply-Add (FMA) instructions.
- Replace repeated division with multiplication by reciprocals.
- Enable SIMD/vector instructions.
- Cache repeated computations.
- Reduce unnecessary conversions.
- Enable compiler optimizations.

Detect Overflow

IEEE 754 overflow occurs when:

- The computed value exceeds the maximum representable value.
  Example:
  Result > 1.7976931348623157 × 10^308
  Then:
  Result = +∞ or −∞
  Overflow Flag = Set
  Also check for:
- Underflow (very small values become 0)
- NaN (invalid operations like 0/0)

Result

Using appropriate precision, optimized arithmetic operations, and

efficient hardware features improves execution speed while maintaining the required numerical accuracy.